

1. FCFS Scheduling algorithm

```
GNU nano 8.7.1
#include <stdio.h>

int main()
{
    int n, i;
    int bt[20], wt[20], tat[20];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time for each process:\n");

    for (i = 0; i < n; i++)
    {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // Waiting time
    wt[0] = 0;

    for (i = 1; i < n; i++)
    {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Turnaround time
    for (i = 0; i < n; i++)
    {
        tat[i] = wt[i] + bt[i];
    }

    printf("\nProcess\tBT\tWT\tTAT\n");

    for (i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\n",
            i + 1, bt[i], wt[i], tat[i]);
    }

    return 0;
}
```

OUTPUTSCREENSHOT:

```
sabiya@LAPTOP-DOV34HMC:~$ nano FCFS.c
sabiya@LAPTOP-DOV34HMC:~$ gcc FCFS.c -o shell
sabiya@LAPTOP-DOV34HMC:~$ ./shell
Enter number of processes: 2
Enter burst time for each process:
P1: 5
P2: 3
Process BT      WT      TAT
P1      5        0        5
P2      3        5        8
sabiya@LAPTOP-DOV34HMC:~$ |
```

2. SJF_NONpreemptive

```
GNU nano 8.7.1
#include <stdio.h>

int main()
{
    int n, i, j;
    int bt[20], wt[20], tat[20];
    int temp;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst times:\n");

    for (i = 0; i < n; i++)
    {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // Sort burst times
    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (bt[i] > bt[j])
            {
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }

    // Waiting time
    wt[0] = 0;

    for (i = 1; i < n; i++)
    {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Turnaround time
    for (i = 0; i < n; i++)
    {
        tat[i] = wt[i] + bt[i];
    }

    printf("\nProcess\tBT\tWT\tTAT\n");

    for (i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\n",
            i + 1, bt[i], wt[i], tat[i]);
    }

    return 0;
}
```

OUTPUTSCREENSHOT:

```
sabiya@LAPTOP-DOV34WDC:~$ nano SJFNonPreemptive.c
sabiya@LAPTOP-DOV34WDC:~$ gcc SJFNonPreemptive.c -o shell
sabiya@LAPTOP-DOV34WDC:~$ ./shell
Enter number of processes: 2
Enter burst times:
P1: 3
P2: 5

Process BT      WT      TAT
P1      3        0        3
P2      5        3        8
sabiya@LAPTOP-DOV34WDC:~$ |
```

3. Priority

```
GNU nano 8.7.1
#include <stdio.h>

int main() {
    int n, i, j;
    int bt[20], priority[20], wt[20], tat[20];
    int temp;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time and priority:\n");

    for (i = 0; i < n; i++) {
        printf("P%d Burst Time: ", i + 1);
        scanf("%d", &bt[i]);

        printf("P%d Priority: ", i + 1);
        scanf("%d", &priority[i]);
    }

    // Sort according to priority
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (priority[i] > priority[j]) {
                temp = priority[i];
                priority[i] = priority[j];
                priority[j] = temp;

                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }

    // Waiting time
    wt[0] = 0;

    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    printf("\nProcess\tBT\tPriority\tWT\tTAT\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\n",
            i + 1, bt[i], priority[i], wt[i], tat[i]);
    }

    return 0;
}
```

OUTPUTSCREENSHOT:

```
sabiyah@LAPTOP-DOVJSHMC:~$ nano priority.c
sabiyah@LAPTOP-DOVJSHMC:~$ gcc priority.c -o shell
sabiyah@LAPTOP-DOVJSHMC:~$ ./shell
Enter number of processes: 3
Enter burst time and priority:
P1 Burst Time: 2
P1 Priority: 5
P2 Burst Time: 4
P2 Priority: 3
P3 Burst Time: 2
P3 Priority: 2
Process BT    Priority    WT    TAT
P1    2    2    6    2
P2    4    3    2    6
P3    2    5    6    8
sabiyah@LAPTOP-DOVJSHMC:~$
```

4.ROUND ROBIN

```
GNU nano 8.7.1
#include <stdio.h>

int main() {
    int n, i;
    int bt[20], rem_bt[20];
    int wt[20], tat[20];
    int quantum;
    int time = 0;
    int done;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time:\n");

    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);

        rem_bt[i] = bt[i];
        wt[i] = 0;
    }

    printf("Enter time quantum: ");
    scanf("%d", &quantum);

    // Round Robin
    while (1) {
        done = 1;

        for (i = 0; i < n; i++) {
            if (rem_bt[i] > 0) {
                done = 0;

                if (rem_bt[i] > quantum) {
                    time = time + quantum;
                    rem_bt[i] = rem_bt[i] - quantum;
                }
                else {
                    time = time + rem_bt[i];

                    wt[i] = time - bt[i];

                    rem_bt[i] = 0;
                }
            }
        }
    }
}
```

```

        if (done == 1)
            break;
    }

    // Turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    printf("\nProcess\tBT\tWT\tTAT\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n",
            i + 1, bt[i], wt[i], tat[i]);
    }

    return 0;
}
|

```

OUTPUTSCREENSHOT:

```

sabiya@LAPTOP-DOV34WDC:~$ nano RoundRobyn.c
sabiya@LAPTOP-DOV34WDC:~$ gcc RoundRobyn.c -o shell
sabiya@LAPTOP-DOV34WDC:~$ ./shell
Enter number of processes: 3
Enter burst time:
P1: 7
P2: 6
P3: 5
Enter time quantum: 2

Process BT      WT      TAT
P1      7       11      18
P2      6       10      16
P3      5       12      17
sabiya@LAPTOP-DOV34WDC:~$

```